

Phase-1: Brainstorming & Concept Inception

Identifying security vulnerability patterns, data isolation boundaries, and granular authorization models.

Problem Statement & Context

Within multi-department educational or enterprise platforms, academic and operational data is routinely consolidated within centralized custom tables. In default configurations, standard role assignments fail to partition records dynamically based on field values, creating severe risks of horizontal privilege escalation, data leakage, and compliance failures.

The Out-of-the-Box Security Gap

Traditional table-level roles provide blanket visibility: once a user possesses read access, they can query and inspect rows across all institutional branches (e.g., ECE, CSE, and EEE).

Proposed Script-Controlled Guardrail

Enforce an intelligent runtime evaluation barrier directly within the platform's Access Control List (ACL) layer, dynamically filtering rows according to branch affiliations.

Core Ideation Themes & Design Criteria

Zero-Trust Data Isolation

No user assumes implicit access. Academic data belonging to the EEE division must be completely imperceptible to personnel from other branches.

Decoupled CRUD Roles

Rather than combining permissions under a single monolithic role, Read, Create, Write, and Delete rights are segregated into individual, auditable roles.

Administrative Governance

System Administrators must retain global oversight across all branches via automated platform overrides without breaking custom runtime scripts.

Strategic Architecture Decision Tree

A Custom Table vs. Core Extension

Selected a dedicated standalone table (`u_institution_details`) in the Global Application Scope to prevent schema pollution of standard task tables.

B Business Rule vs. ACL Enforcement

Selected Contextual ACLs over before query Business Rules to guarantee rigorous server-side enforcement across UI forms, list views, and REST API endpoints.

C Three-Tier Evaluation Model

Decided to enforce security using the complete ServiceNow triad: Assigned Roles + Data Conditions + Advanced Server-Side JavaScript.

Phase Outcome: Formalized project charter to establish a 3-hour micro-project implementing script-controlled ACLs on student records with departmental row isolation.

Phase-2: Requirement Analysis & Specification

Translating brainstorming objectives into rigorous functional, non-functional, and compliance requirements.

Functional Access Requirements

Access requirements dictate the operational behavior across all four CRUD primitives for both authenticated branch users and elevated platform administrators.

Operation	Target User / Persona	Requirement Specification	Expected UI Manifestation
READ	EEE Department User	Must only view records where Branch == 'EEE'. Records with ECE and CSE branch tags must be suppressed.	List view filters out unauthorized branches; form links restricted.
READ	System Administrator	Must maintain full global read visibility across ECE, EEE, and CSE departments for auditing purposes.	All rows visible without any row-count truncation banners.
CREATE	Branch User with Role	Record creation must be forbidden unless the user is explicitly granted the designated record creation role.	The <i>New</i> button in the list header is dynamically shown/hidden.
WRITE	Branch User with Role	Modifications to fields (such as Description, Email, and Phone) are prohibited without designated edit rights.	Double-click list cell edit disabled; form inputs rendered read-only.
DELETE	Branch User with Role	Record purging rights must be isolated to prevent accidental deletion of institutional student data.	<i>Delete</i> action removed from contextual menus and list controls.

Non-Functional & Platform Security Constraints

Performance & Latency Constraint

ACL scripts evaluate per record on list view queries. The JavaScript snippet must execute in < 2ms without heavy GlideRecord database round-trips.

Session Elevation Protocol

Modification of security rules within `sys_security_acl` is strictly gated behind the temporary `security_admin` role elevation lifecycle.

Traceability Matrix: Requirements to Technical Artifacts

Requirement ID	Description	Implemented ServiceNow Object
REQ-SEC-01	Field-level restriction on Branch value	Read ACL Condition: <code>u_institution_details.u_branch == 'EEE'</code>
REQ-SEC-02	Scripted role verification	Advanced ACL Script calling <code>gs.hasRole('bb1')</code>
REQ-SEC-03	Granular CRUD access partitioning	Custom roles: bb1 (R), bb2 (C), bb3 (W), bb4 (D)
REQ-SEC-04	Unrestricted administrator oversight	ACL Property: <code>admin_overrides = true</code> + Script admin check

Phase-3: Project & Architecture Design

Defining data models, role hierarchy relationships, and the contextual security evaluation engine.

Data Schema Architecture: u_institution_details

The custom schema is designed to model student demographics, mentor assignments, and departmental taxonomy under the Global Scope.

Column Label	Physical Column	Data Type	Max Length	Default / Reference Target
Student Roll Number	u_student_roll_number	String	40	javascript:global.getNextObjNumberPadded();
Student Name	u_student_name	Reference	32	Table Reference: sys_user
Faculty Name	u_faculty_name	Reference	32	Table Reference: sys_user
Branch	u_branch	Choice	40	Options: ECE, EEE, CSE
Email	u_email	String	40	Standard RFC-5322 string structure
Phone Number	u_phone_number	String	40	International phone formatting string
Description	u_description	String	1,000	Multi-line academic progress summary

Role Assignment & Persona Topology

Role: bb1 Operation: READ Permits browsing records matching the branch condition.	Role: bb2 Operation: CREATE Permits spawning and submitting new student records.	Role: bb3 Operation: WRITE Permits updating and in-line editing of fields.	Role: bb4 Operation: DELETE Permits row-level and bulk record deletion.
---	--	--	---

Contextual Security Evaluation Logic

Evaluation Formula: When an access request occurs, ServiceNow evaluates security rules using a strict logical AND condition across three distinct layers: Access_Granted = (Admin_Override) [User_Has_Role('bb1') && Condition(Branch == 'EEE') && Script_Returns_True()]
--

Phase-4: Project Planning & Work Breakdown

Sprint schedule, resource allocation, milestone dependencies, and risk mitigation strategies.

Work Breakdown Structure (WBS) & Timeline

The 3-hour micro-project was segmented into four discrete, sequential delivery milestones designed for rapid turnaround and zero deployment rework.

Milestone	Milestone Phase	Execution Tasks & Deliverables	Duration	Pre-requisite
M-1	User & Security Roles Setup	Create test user <code>EEE User</code> ; create roles <code>bb1</code> , <code>bb2</code> , <code>bb3</code> , <code>bb4</code> ; configure membership.	30 mins	Admin Access
M-2	Table & Mock Data Staging	Create table <code>u_institution_details</code> ; create 7 schema attributes; ingest 6 test records across branches.	45 mins	M-1
M-3	ACL Architecture Build	Elevate to <code>security_admin</code> ; configure Read (scripted), Create, Write, and Delete ACL entries.	60 mins	M-2
M-4	Persona Testing & Sign-off	Execute progressive testing matrix across roles via session impersonation; document test results.	45 mins	M-3

Resource & Environment Allocation

Instance & Scope Configuration Instance Environment: ServiceNow Personal Developer Instance (PDI) Application Scope: Global Update Set: ACL Project [Global]	Assigned Test Persona User ID: <code>EEEUser</code> Full Name: EEE User Email: <code>eeeuser@gmail.com</code> Roles: Incrementally granted (<code>bb1</code> through <code>bb4</code>)
---	---

Risk Management & Mitigation Framework

Risk: Accidental Platform Lockout Misconfigured table-level Read ACLs can accidentally lock administrators out of custom data records. Mitigation: Ensure <code>admin_overrides = true</code> remains checked on all newly spawned ACLs.	Risk: Role Elevation Expiration Session elevation to <code>security_admin</code> expires automatically upon logout or idle timeout. Mitigation: Re-elevate credentials immediately before saving modifications to the ACL table.
--	--

Phase-5: Project Development & Implementation

Step-by-step configuration of users, roles, custom table schema, and scripted Access Control Lists.

Step 1: Security Elevation & User Provisioning

Navigated to *User Administration* → *Users* and generated the test user account. Next, navigated to *User Administration* → *Roles* and generated custom roles:

Role: bb1
Read Permission

Role: bb2
Create Permission

Role: bb3
Write Permission

Role: bb4
Delete Permission

Step 2: Custom Table Architecture & Data Seeding

Created table `u_institution_details` under *System Definition* → *Tables*. Ingested 6 baseline student records:

Roll Number	Student Name	Faculty Mentor	Branch	Email	Phone
INS0001006	Praneeth	ITIL User	ECE	praneeth@gmail.com	8989090906
INS0001008	Abraham Lincoln	Malla Sravan Kumar	CSE	sravan@gmail.com	8978908989
INS0001005	Srinu Nandipalli	ITIL User	ECE	srinu@gmail.com	9089909090
INS0001007	Guna Sai	Abel Tuter	EEE	saiguna@gmail.com	9090898765
INS0001001	Yaswanth Nikku	Abraham Lincoln	CSE	yaswanth@gmail.com	8989898989
INS0001004	Prashanth Reddy	Abel Tuter	EEE	prashanth@gmail.com	9090909090

Step 3: Access Control List (ACL) Configuration

Elevated to `security_admin` under session profile and navigated to *System Security* → *Access Control (ACL)*:

Read ACL Implementation (Script-Controlled + Data Condition)

Table: `u_institution_details` | Operation: `read` | Requires role: `bb1` | Condition: `Branch is EEE`

```
(function () { // Allow admin users full access if (gs.hasRole('admin')) { return true; } })
```

CRUD Auxiliary ACLs: Configured Create ACL (Requires: `bb2`), Write ACL (Requires: `bb3`), and Delete ACL (Requires: `bb4`) on target `u_institution_details`.

Phase-6: Project Testing & Verification Matrix

Formal test execution across user impersonations, UI validation, and regression confirmation.

Test Methodology & Execution Environment

Verification was performed by impersonating target test accounts within the ServiceNow native interface, navigating directly to `u_institution_details.list`, and inspecting form layouts, action buttons, and contextual menus.

Test ID	Persona & Roles	Expected Acceptance Criteria	Observed Execution Result	Status
TS-01	System Admin admin	Unrestricted visibility over all 6 records across ECE, EEE, and CSE branches.	All 6 records displayed. Full CRUD controls operational.	PASS
TS-02	Unassigned User no roles	Total read denial. Record query returns empty list.	UI reports "No records to display". All buttons suppressed.	PASS
TS-03	EEE User bb1	View only records where Branch == 'EEE'. No record creation allowed.	Only INS0001007 & INS0001004 visible. New button hidden.	PASS
TS-04	EEE User bb1 + bb2	Maintain branch read filtering and render the New button on header.	New button rendered. New record form opens with auto-numbering.	PASS
TS-05	EEE User bb1 + bb2 + bb3	Enable write operations; permit inline editing on list cells and form updates.	Double-click list editing unlocked; updated Description field successfully.	PASS
TS-06	EEE User bb1 + bb2 + bb3 + bb4	Enable record deletion via list selection checkboxes and form delete button.	Delete action visible in dropdown. Record INS0001007 purged cleanly.	PASS

Security Regression Evaluation

Verification of Departmental Masking

Confirmed that records belonging to ECE (INS0001006, INS0001005) and CSE (INS0001008, INS0001001) never appear in the DOM, network payloads, or export batches under the EEE User session.

Administrative Override Validation

Confirmed that logging back in as System Administrator immediately restored visibility to all 6 table records, validating that `admin_overrides` functioned as intended.

Testing Conclusion: 100% of test cases passed on the initial validation run with zero security bypass anomalies detected.

Phase-7: Project Technical Documentation

Platform architectural mechanics, security engine behavior, and long-term maintenance runbook.

Contextual Security Architecture Deep Dive

Access Control Lists (ACLs) are the core gatekeeper of ServiceNow data integrity. Understanding how the evaluation engine processes incoming requests prevents architectural vulnerabilities.

The ACL Evaluation Pipeline

For any user query or form request, ServiceNow evaluates rules in strict order:

- 1 Table-Level ACL Check**
Checks if the user has access to the table as a whole (`u_institution_details`). If failed, access is denied immediately.
- 2 Field-Level ACL Check**
Checks if the user has permission to read or modify specific columns (`u_institution_details.*` or specific fields).
- 3 Rule Triad Evaluation**
Within each ACL rule, all three conditions must evaluate to true: *Role Check* && *Condition Match* && *Script Execution*.

Administrator Maintenance Runbook

Maintenance Scenario	Recommended Procedure	Safety Precautions
Adding New Branches (e.g., MECH)	Add choice option to <code>u_branch</code> field dictionary; create new role (e.g., <code>bb_mech</code>); extend ACL script condition logic.	Do not modify baseline admin bypass logic in script.
Debugging Access Issues	Navigate to <i>System Diagnostics</i> → <i>Session Debug</i> → <i>Debug Security Rules</i> ; reload target list view.	Disable debugger immediately after troubleshooting to preserve session speed.
Promoting to Production	Capture all dictionary changes, custom roles, and ACLs in an explicitly named Global Update Set.	Verify update set does not contain accidental user test records.

Governance Best Practices Derived

- Role Granularity:** Never lump multiple CRUD capabilities into a single operational role for custom tables.
- Server-Side Primacy:** Do not rely on UI Policies or Client Scripts for data protection, as they can be easily bypassed via the browser console or REST queries.

Phase-8: Project Demonstration & Evidence

Empirical validation, session impersonation walkthrough, and live platform evidence capture.

Live Demonstration Walkthrough

The functional capability of the script-controlled ACL implementation was verified and documented across three distinct administrative session personas:

Session Persona A: EEE User with Full Roles (bb1 + bb2 + bb3 + bb4)

Impersonated EEE User and loaded `u_institution_details.list`:

Roll Number	Student Name	Branch	Description	Email	Phone
INS0001007	Guna Sai	EEE	ACL is Executed	saiguna@gmail.com	9090898765
INS0001004	Prashanth Reddy	EEE	(empty)	prashanth@gmail.com	9090909090

✓ Evidence: Only EEE rows displayed. "New" button enabled. Inline field updates saved successfully. Row deletion confirmed via "Actions on selected rows... → Delete".

Session Persona B: Unprivileged User Session

Impersonated user without bb1 role and loaded the table list:

"No records to display" • Create / Edit / Delete UI buttons suppressed

Session Persona C: System Administrator Session

Navigated as System Administrator to verify full governance bypass:

All 6 original records (INS0001001 through INS0001008) across ECE, EEE, and CSE branches visible simultaneously. Admin override mechanism functioned seamlessly without any manual script intervention.

Final Project Sign-Off:

All 8 project lifecycle phases—from ideation and requirements mapping to architecture design, implementation, and multi-persona validation—have been executed to completion. The implementation provides airtight record-level security in strict alignment with ServiceNow enterprise security standards.